

For each category, we strive to break it down into atomic and orthogonal labels, but in practice this is hard to achieve, so predicting “multiple labels” is recommended.

Coverage and Diversity Guarantee

For the Cartesian product of `code_lang` × `task_type` × application, ensure that each combination (of the categories marked with ★★★) has at least **5** tasks.

code_lang

★★★ marks **required**; the rest are optional.

```
**Available Language Labels:**
- ★★★**go**: Go/Golang
- ★★★**c**: C
- ★★★**c++**: C++
- ★★★**python**: Python
- ★★★**java**: Java
- ★★★**js/ts**: JavaScript or TypeScript
- ★★★**php**: PHP
- ★★★**shell**: shell/bash
- ★★★**c#**: C# (.NET)
- ★★★**html/css**: HTML, CSS, or primarily web markup/styling
- ★★★**sql**: SQL (database queries)
- ★★★**rust**: Rust
- **kotlin**: Kotlin
- **swift**: Swift
- **ruby**: Ruby
- **lua**: Lua
- **other**: Any other programming language not listed above
- **mixed**: Multiple languages are significantly used (when no
single language dominates)
- **unknown**: Unknown language, or language not specified, or
cannot infer the language from the task statement
```

task_type

Granularity down to second-level labels.

★★★ marks **required**; the rest are optional.

```
## Fix
Correcting erroneous behavior, faults, or regression issues.
Primary goal is to restore correctness/stability, not to add new
features.

- ★★★**bug-fix**: Fix functional defects (incorrect output,
crashes, exceptions, memory leaks, etc.)
- **performance-fix**: Fix performance issues treated as defects
(timeouts, severe slowdowns, etc.)
- ★★★**compatibility-fix**: Fix incompatibilities across
versions/platforms/browsers

## Implementation
Adding or extending product capabilities. Primary goal is new
functionality or newly supported behavior.
```

- *****feature****: Introduce new externally visible functionality/capability
- ****enhancement****: Improve or extend existing functionality (non-trivial capability improvements)
- ****backward-compatibility****: Add compatibility path for old clients/versions (dual read/write, fallback solutions)
- ****from_scratch****: Completely generate code from scratch without modifying existing codebase (creating entirely new files/modules)

Testing & Quality

Primary work is adding/fixing tests or improving quality gates.

- *****test-add****: Add missing unit/integration/end-to-end tests
- ****test-fix****: Fix failing tests (assertions, mocks, fixtures)
- *****coverage****: Improve test coverage and fill gaps
- ****debug-support****: Add debugging/diagnostic hooks (feature flags, debug logging, trace switches)

Refactoring & Maintenance

Improving code structure, maintainability, or efficiency with minimal changes to external behavior (unless explicitly stated).

- *****refactor****: Restructure architecture: modularization, layering, interface redesign, decoupling
- ****style****: Improve code quality: reduce duplication/complexity, enhance readability, improve naming, formatting, linter fixes, type-checking compliance
- ****deprecation****: Deprecate old interfaces/paths; add warnings and migration tools

Build & Release

Changes to build, CI/CD, packaging, deployment, configuration, or developer tools. Primary outcome is improving build/release reliability or reproducibility.

- *****build-fix****: Fix compilation/build failures (linking, packaging errors)
- *****ci-fix****: Fix CI pipeline failures or flakiness (tasks, runners, caching)
- ****deployment****: Deployment/release pipeline changes (Helm, Terraform, release strategies)
- ****packaging****: Produce and publish artifacts (Docker images, wheels, npm packages)
- ****config****: Environment/configuration management (config schemas, secret connections, defaults)

Documentation & Design

Primary work is producing/updating documentation or design artifacts for developers or stakeholders.

- ****user-docs****: README, API documentation, tutorials, usage guides, getting started documentation
- ****design-spec****: Design documents, specifications, RFCs, ADRs, PRDs describing requirements, architecture, and trade-offs
- ****diagram****: Architecture diagrams/flowcharts/interface illustrations

Security & Compliance

Primary goal is security hardening or compliance (privacy, auditability, supply chain).

- ****security-fix****: Discover and fix vulnerabilities
- ****auth****: Authentication/authorization changes
- ****privacy****: Data privacy, retention, auditing, access control

Non_Coding

Non-coding tasks that do not involve programming or technical implementation. Examples include:

- General questions and discussions
- Conceptual explanations without code implementation
- Requirements gathering or planning discussions
- Pure documentation review or writing (without technical implementation)
- Brainstorming or idea exploration

application

Granularity down to first-level labels.

*** Client_UI

- ****Core****: User-facing interface, interaction design, and accessibility
- ****Includes****:
 - ****Web Frontend****: React, Vue, Angular, HTML/CSS/JS, DOM manipulation
 - ****Mobile****: iOS (Swift/Obj-C), Android (Kotlin/Java), Flutter, React Native
 - ****Desktop GUI****: Electron, Qt, WPF, Swing, PyQt
 - ****Boundary****: Code that runs on the client-side or handles visual rendering/user events

*** Backend_Infrastructure

- ****Core****: Server-side application development and the infrastructure/platform services needed to deploy, run, and operate these applications
- ****Includes****:
 - ****Backend Development****:
 - API Layer: RESTful/GraphQL APIs (Spring Boot, Django, Express, FastAPI, Gin)
 - Request Handling: Routing, middleware, request/response processing
 - Authentication & Authorization: Auth services, session management, access control
 - Business Logic: Application-level business rules and workflows
 - Data Access: CRUD operations, database I/O, ORM interactions
 - Integration: Message queues, microservices communication, third-party service integration
 - ****Infrastructure & Operations****:
 - Cloud & Containers: AWS/GCP/Azure, Docker, Kubernetes, Helm
 - Infrastructure as Code: Terraform, CloudFormation, Ansible
 - CI/CD: Jenkins, GitLab CI, GitHub Actions, ArgoCD
 - Observability: Monitoring (Prometheus, Grafana), logging (ELK), tracing (Jaeger)
 - Platform Services: Service mesh, API gateways, secrets management (Vault)
 - ****Boundary****:
 - Code that handles HTTP requests, routing, database I/O, or orchestrates business workflows
 - Infrastructure provisioning, deployment automation, and operational tooling

- ****Excludes****: Complex domain-specific algorithms independent of the web layer (consider `Business_Domain_Logic` instead)

***** AI_ML**

- **Core**: Machine learning model development, training, fine-tuning, inference optimization
- **Includes**:
 - **Data Processing**: feature engineering, data pipelines for ML, ETL/ELT workflows, batch/stream processing (Spark, Flink, Kafka) for model training/inference, feature stores, and data preprocessing at scale.
 - **Infra**: Model architecture (Transformers, CNNs), MLOps, experiment tracking, AI frameworks (PyTorch, TensorFlow, scikit-learn)
 - **NLP/CV/RL applications**
 - **Data analysis**: Statistical analysis, data cleaning, data manipulation, and visualization.

***** Database_Storage**

- ****Core****: Data persistence, storage architecture, and data management
- ****Includes****:
 - ****Design & Modeling****: Database schema design, ER diagrams, normalization strategies
 - ****Performance Tuning****: Query optimization, indexing strategies, caching (Redis/Memcached)
 - ****Database Operations****: MySQL/PostgreSQL/MongoDB configuration, migrations, backup, sharding
 - ****Data Warehousing****: Snowflake, Redshift, data lakes, ETL/ELT pipelines
- ****Boundary****: Work involving data storage, querying, and persistence

***** Business_Domain_Logic**

- **Core**: Pure business rules, state management, and enterprise logic independent of UI or Infrastructure.
- **Includes**:
 - Complex workflows, state machines, booking/reservation systems.
 - Financial calculations, fintech systems, inventory management algorithms, compliance rule engines.
 - Specific business scenarios (e.g., medical data processing) distinct from generic CRUD.

Graphics_Media

- **Core**:
- **Includes**:
 - **Game development**: game engine development and scripting (Unity, Unreal, Godot),
 - **Rendering**: real-time and offline rendering (OpenGL, Vulkan, DirectX, Metal), shader programming, 3D modeling and animation, physics simulation
 - **Multimedia**: Audio/Video transcoding (FFmpeg), Codec implementation, Image manipulation (OpenCV), AR/VR.

Embedded_Systems

Hardware interaction, real-time systems, IoT devices, firmware development, microcontroller programming (Arduino, ESP32, STM32), RTOS, device drivers, sensor integration, communication protocols (I2C, SPI, UART, CAN), power management, and low-level system programming.

Operating_System

OS kernel development, system calls, process/thread management, memory management, file systems, device drivers, scheduling algorithms, inter-process communication, virtualization, bootloaders, and OS internals (Linux, Windows, macOS, RTOS).

Scientific_Computing

Numerical analysis, scientific simulations, computational physics/chemistry/biology, linear algebra libraries (NumPy, BLAS, LAPACK), finite element analysis, molecular dynamics, HPC (high-performance computing), parallel computing (MPI, OpenMP), climate modeling, and scientific visualization.

Other

Tasks that do not clearly fit any of the above domains, or span multiple domains without a clear primary focus.

- other dimension

- difficulty: how difficult the problem is
- user_spec: user specification is specific or adhoc
- verifiable or not: